



C++ Program For Fibonacci Numbers

Last Updated : 27 May, 2026



The Fibonacci series is a sequence in which each number is the sum of the previous two numbers. The series starts with 0 and 1, and the remaining terms are generated using the relation:

$$F(n)=F(n-1)+F(n-2)$$

where:

- $F(0) = 0$
- $F(1) = 1$

Example:

Input: 5

Output: 5

Explanation: In the Fibonacci series 0, 1, 1, 2, 3, 5, 8, 13..., the value at index 5 is 5 (using 0-based indexing).

Different Ways to Find Nth Fibonacci Number in C++

There are multiple approaches to find the nth Fibonacci number in C++. Each method has different time and space complexities.

Using Recursion

The simplest way to find the nth Fibonacci number is by using recursion. In this method, the function repeatedly calls itself to calculate previous Fibonacci numbers.

Approach:

- If n is 0 or 1, return n .
- Otherwise, recursively calculate: $F(n)=F(n-1)+F(n-2)$

```
#include <bits/stdc++.h>
using namespace std;

int fib(int n) {

    // If n is 1 or 0, then return n,
    // works for 0th and 1st terms
    if (n <= 1)
        return n;

    // Recurrence relation to find
    // the rest of the terms
```

```

        return fib(n - 1) + fib(n - 2);
    }

    int main() {
        int n = 5;

        // Finding nth term
        cout << fib(n);
        return 0;
    }

```



Output

5

Explanation: The function recursively calculates the Fibonacci value by adding the previous two Fibonacci numbers.

- **Time Complexity:** $O(2n)$
- **Auxiliary Space:** $O(n)$

Optimization of Recursion Method

The recursive solution can be optimized by passing the previous two Fibonacci numbers as parameters. This avoids repeated calculations.

Approach:

- Store the previous two Fibonacci numbers.
- Update them during recursive calls.
- Avoid recalculating already computed values.

```

#include <iostream>
using namespace std;

// Helper function to find the
// nth fibonacci number using recursion
int fibHelper(int n, int prev2, int prev1) {

    // When n reaches 0, return prev2

```



[DSA](#)
[Practice Problems](#)
[C](#)
[C++](#)
[Java](#)
[Python](#)
[JavaScript](#)
[Data Science](#)
[Machine Learning](#)
[Courses](#)

[Sign In](#)

```

        return prev2,
    }

    // When n reaches 1, return prev1
    if (n == 1) {
        return prev1;
    }

    // Recursive call with updated
    // parameters to find rest
    // of the fibonacci numbers

```

```

        return fibHelper(n - 1, prev1, prev2 + prev1);
    }

    int fib(int n) {

        // Calling recursive function
        return fibHelper(n, 0, 1);
    }

    int main() {
        int n = 5;

        // Finding the nth Fibonacci number
        cout << fib(n);
        return 0;
    }

```

Output

5

Explanation: Instead of recalculating previous Fibonacci numbers, the function carries forward the last two values in each recursive call.

- **Time Complexity:** $O(n)$
- **Auxiliary Space:** $O(n)$

Using Loops

The iterative approach using loops is one of the most efficient and commonly used methods.

Approach:

- Maintain two variables for previous Fibonacci numbers.
- Use a loop to generate the next Fibonacci term.
- Continue until the nth term is reached.

```

#include <bits/stdc++.h>
using namespace std;

int fib(int n) {

    // For 0th and 1st term
    if (n <= 1)
        return n;

    // Variable to store the
    // last two terms
    int prev1 = 1, prev2 = 0;

    // Variable that stores the
    // current fibonacci term

```

```

int curr;

// Calculating the next
// fibonacci number by using
// the previous two number
for (int i = 2; i <= n; i++) {
    curr = prev1 + prev2;
    prev2 = prev1;
    prev1 = curr;
}
return curr;
}

int main() {
    int n = 5;
    cout << fib(n);
    return 0;
}

```

Output

5

Explanation: The loop continuously updates the last two Fibonacci numbers until the required term is found.

- **Time Complexity:** $O(n)$
- **Auxiliary Space:** $O(1)$

Using Matrix Exponentiation

In this approach, the Fibonacci series is represented using a transformation matrix based on the Fibonacci relation:

$$F(n) = F(n-1) + F(n-2)$$

where:

- $F(0) = 0$
- $F(1) = 1$

This method efficiently calculates the n th Fibonacci number by raising the transformation matrix to the power $(n - 1)$. Matrix exponentiation uses the divide-and-conquer technique to compute powers quickly, making it much faster for large values of n .

```

#include <bits/stdc++.h>
using namespace std;

// Function to multiply 2 * 2 matrix
void multiply(vector<vector<int>> &m1,
             vector<vector<int>> &m2) {

    // Matrix to store result

```

```

vector<vector<int>>> res(2, vector<int>(2));
res[0][0] = m1[0][0] * m2[0][0]
           + m1[0][1] * m2[1][0];
res[0][1] = m1[0][0] * m2[0][1]
           + m1[0][1] * m2[1][1];
res[1][0] = m1[1][0] * m2[0][0]
           + m1[1][1] * m2[1][0];
res[1][1] = m1[1][0] * m2[0][1]
           + m1[1][1] * m2[1][1];

// Copying Multiplied matrix to m1
m1[0][0] = res[0][0];
m1[0][1] = res[0][1];
m1[1][0] = res[1][0];
m1[1][1] = res[1][1];
}

// Calculating the power(mat,n) in
// log n time using matrix exponentiation
void power(vector<vector<int>>> &m, int n) {

    // Base case
    if(n == 0 || n == 1)
        return;

    // Identity matrix
    vector<vector<int>>> i = {{1, 1},
                           {1, 0}};

    power(m, n / 2);
    multiply(m, m);

    if (n % 2 != 0)
        multiply(m, i);
}

int fib(int n) {

    // For 0th and 1st term
    if (n < 2)
        return n;

    // Representing fibonacci series
    // as matrix
    vector<vector<int>>> m = {{1, 1},
                           {1, 0}};

    // Finding (n - 1)th power of
    // matrix m
    power(m, n - 1);

    // Returning nth fibonacci number
    return m[0][0];
}

```

```
int main() {
    int n = 5;
    cout << fib(n);
    return 0;
}
```

Output

5

Explanation: Matrix exponentiation reduces repeated calculations by using divide-and-conquer techniques.

- **Time Complexity:** $O(\log n)$
- **Auxiliary Space:** $O(\log n)$

 Comment

 kartik

 3



Explore

C++ Basics



Core Concepts



OOP



Standard Template Library(STL)



Practice & Problems



Courses



Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GFG
Corporate Solution

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

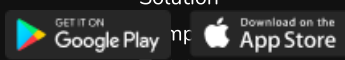
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design



Training Program